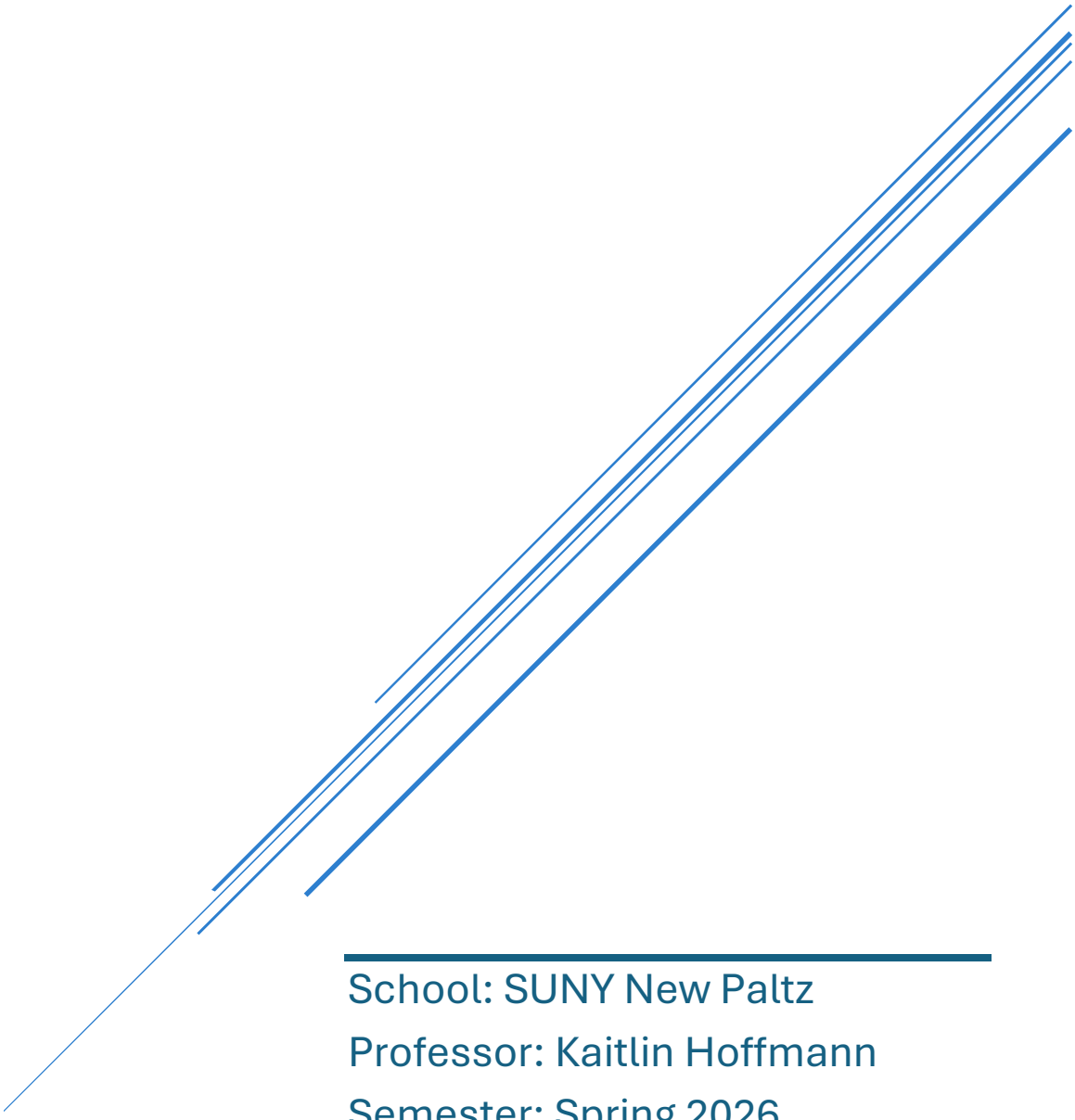
ENV AND SET-UID LAB

Student: Jennifer Elie
Course: CPS 593 Cybersecurity
Due: March 26th 2026



School: SUNY New Paltz
Professor: Kaitlin Hoffmann
Semester: Spring 2026

0. VM Setup:

0.1. Check the system for updates

Command: sudo apt update

Command explanation:

- **Sudo:** Temporarily allows administrator-level permission for command execution.
- **Apt:** Stands for Advanced Package Tool, and is used to manage packages on the system.
- **Update:** This keyword checks the repositories of the packages already on the system for available updates, and creates a list of these packages.

Command line output:

```
jelie@Jelie-VM:~$ sudo apt update
[sudo] password for jelie:
Hit:1 http://us.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu jammy-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
21 packages can be upgraded. Run 'apt list --upgradable' to see them.
jelie@Jelie-VM:~$
```

0.2. Upgrade the system

Command: sudo apt upgrade

Command explanation:

- **Sudo:** Temporarily allows administrator-level permission for command execution.
- **Apt:** Stands for Advanced Package Tool, and is used to manage packages on the system.
- **upgrade:** This keyword upgrades installed packages to their latest available versions, using the list created during the update command.

Command line output:

```
jelie@Jelie-VM:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following NEW packages will be installed:
  linux-headers-6.8.0-106-generic linux-hwe-6.8-headers-6.8.0-106
  linux-hwe-6.8-tools-6.8.0-106 linux-image-6.8.0-106-generic
  linux-modules-6.8.0-106-generic linux-modules-extra-6.8.0-106-generic
  linux-tools-6.8.0-106-generic
The following packages will be upgraded:
  coreutils gir1.2-nm-1.0 libexiv2-27 libnftables1 libnm0 libssh-4
  linux-generic-hwe-22.04 linux-headers-generic-hwe-22.04
  linux-image-generic-hwe-22.04 linux-tools-common network-manager
  network-manager-config-connectivity-ubuntu network-manager-openvpn
  network-manager-openvpn-gnome nftables python3-cryptography snapd vim-common
  vim-tiny wpasupplicant xxd
21 upgraded, 7 newly installed, 0 to remove and 0 not upgraded.
11 standard LTS security updates
Need to get 40.9 MB/184 MB of archives.
After this operation, 769 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us.archive.ubuntu.com/ubuntu jammy-updates/main amd64 coreutils amd
64 8.32-4.1ubuntu1.3 [1,437 kB]
```

0.3. Install the ZSH package

Command: sudo apt install zsh

Command explanation:

- **Sudo:** Temporarily allows administrator-level permission for command execution.
- **Apt:** Stands for Advanced Package Tool, and is used to manage packages on the system.
- **Install:** This keyword installs a specified package onto the system.
- **Zsh:** This argument specifies the Z Shell package being installed.

Command line output:

```
jelie@Jelie-VM:~$ sudo apt install zsh
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-headers-6.8.0-100-generic linux-hwe-6.8-headers-6.8.0-100
  linux-hwe-6.8-tools-6.8.0-100 linux-image-6.8.0-100-generic
  linux-modules-6.8.0-100-generic linux-modules-extra-6.8.0-100-generic
  linux-tools-6.8.0-100-generic
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  zsh-common
Suggested packages:
  zsh-doc
The following NEW packages will be installed:
  zsh zsh-common
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 4,794 kB of archives.
After this operation, 18.2 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://us.archive.ubuntu.com/ubuntu jammy/main amd64 zsh-common all 5.8.1-
1 [3,985 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu jammy/main amd64 zsh amd64 5.8.1-1 [80
9 kB]
```

0.4. Install the Build essential packege

Command: sudo apt-get install build-essential

Command explanation:

- **Sudo:** Temporarily allows administrator-level permission for command execution.
- **Apt-get:** This is the mostly deprecated version of the apt command. It is similar to the other version but does not seem to install updated dependencies like apt alone does. It is not entirely clear why this is being used over apt install for this command.
- **Install:** This keyword installs a specified package onto the system.
- **build-essential:** This is a meta package that includes many other packages, including tools required for compiling software, such as compilers and libraries.

Command line output:

```
jelie@Jelie-VM:~$ sudo apt-get install build-essential
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-headers-6.8.0-100-generic linux-hwe-6.8-headers-6.8.0-100
  linux-hwe-6.8-tools-6.8.0-100 linux-image-6.8.0-100-generic
  linux-modules-6.8.0-100-generic linux-modules-extra-6.8.0-100-generic
  linux-tools-6.8.0-100-generic
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu dpkg-dev fakeroot g++
  g++-11 gcc gcc-11 libalgorithm-diff-perl libalgorithm-diff-xs-perl
  libalgorithm-merge-perl libasan6 libbinutils libc-dev-bin libc-devtools
  libc6-dev libcc1-0 libcrypt-dev libctf-nobfd0 libctf0 libdpkg-perl
  libfakeroot libfile-fcntllock-perl libgcc-11-dev libitm1 liblsan0 libnsl-dev
  libquadmath0 libstdc++-11-dev libtirpc-dev libtsan0 libubsan1 linux-libc-dev
  lto-disabled-list make manpages-dev rpcsvc-proto
Suggested packages:
  binutils-doc debian-keyring g++-multilib g++-11-multilib gcc-11-doc
  gcc-multilib autoconf automake libtool flex bison gcc-doc gcc-11-multilib
  gcc-11-locales glibc-doc git bzr libstdc++-11-doc make-doc
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential dpkg-dev
```

0.5. Install ncal

Command: sudo apt install ncal

Command explanation:

- **Sudo:** Temporarily allows administrator-level permission for command execution.
- **Apt:** Stands for Advanced Package Tool, and is used to manage packages on the system.
- **Install:** This keyword installs a specified package onto the system.
- **Ncal:** This argument specifies the ncal package, which is used to display a calendar in the terminal.

Command line output:

```
jelie@Jelie-VM:~$ sudo apt install ncal
[sudo] password for jelie:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  linux-headers-6.8.0-100-generic linux-hwe-6.8-headers-6.8.0-100
  linux-hwe-6.8-tools-6.8.0-100 linux-image-6.8.0-100-generic
  linux-modules-6.8.0-100-generic linux-modules-extra-6.8.0-100-generic
  linux-tools-6.8.0-100-generic
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  ncal
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 20.2 kB of archives.
After this operation, 69.6 kB of additional disk space will be used.
Get:1 http://us.archive.ubuntu.com/ubuntu jammy/universe amd64 ncal amd64 12.1.7
+nmu3ubuntu2 [20.2 kB]
Fetched 20.2 kB in 0s (326 kB/s)
Selecting previously unselected package ncal.
(Reading database ... 247956 files and directories currently installed.)
Preparing to unpack .../ncal_12.1.7+nmu3ubuntu2_amd64.deb ...
Unpacking ncal (12.1.7+nmu3ubuntu2) ...
Setting up ncal (12.1.7+nmu3ubuntu2) ...
```

1. Overview:

The learning objective of this lab is for students to understand how environment variables affect program and system behaviors. Environment variables are a set of dynamic named values that can affect the way running processes will behave on a computer. They are used by most operating systems, since they were introduced to Unix in 1979. Although environment variables affect program behaviors, how they achieve that is not well understood by many programmers. As a result, if a program uses environment variables, but the programmer does not know that they are used, the program may have vulnerabilities. In this lab, students will understand how environment variables work, how they are propagated from parent process to child, and how they affect system/program behaviors. We are particularly interested in how environment variables affect the behavior of Set-UID programs, which are usually privileged programs. This lab covers the following topics:

- Environment variables
- Set-UID programs
- Securely invoke external programs
- Capability leaking
- Dynamic loader/linker

Readings and videos. Detailed coverage of the Set-UID mechanism, environment variables, and their related security problems can be found in the following:

- Chapters 1 and 2 of the SEED Book, Computer & Internet Security: A Hands-on Approach, 2nd Edition, by Wenliang Du. See details at https://www.handsonsecurity.net.
- Section 2 of the SEED Lecture at Udemy, Computer Security: A Hands-on Approach, by Wenliang Du. See details at https://www.handsonsecurity.net/video.html.

Lab environment. This lab has been tested on the SEED Ubuntu 20.04 VM. You can download a pre-built image from the SEED website, and run the SEED VM on your own computer. However, most of the SEED labs can be conducted on the cloud, and you can follow our instruction to create a SEED VM on the cloud.

2. Lab Tasks:

Files needed for this lab are included in Labsetup.zip, which can be downloaded from the lab's website.

Task 1: Manipulating Environment Variables

In this task, we study the commands that can be used to set and unset environment variables. We are using Bash in the seed account. The default shell that a user uses is set in the /etc/passwd file (the last field of each entry). You can change this to another shell program using the command chsh (please do not do it for this lab). Please do the following tasks:

Command: printenv

Command: env

Command explanation:

- **Printenv:** This command displays the values of environment variables. If no variable is specified, it prints all environment variables.
- **Env:** This command also displays environment variables and can manage them as well.

Command line output:

Here, the printenv command is used. With no arguments it prints every env.

```
jelie@Jelie-VM:~$ printenv
SHELL=/bin/bash
SESSION_MANAGER=local/Jelie-VM:@/tmp/.ICE-unix/1283,unix/Jelie-VM:/tmp/.ICE-unix
/1283
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
SSH_AGENT_LAUNCHER=gnome-keyring
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
GTK_MODULES=gail:atk-bridge
PWD=/home/jelie
LOGNAME=jelie
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=wayland
SYSTEMD_EXEC_PID=1310
XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.RVE6M3
HOME=/home/jelie
USERNAME=jelie
IM_CONFIG_PHASE=1
```

Here, the env command is used. With no flags or arguments it displays every env.

```
jelie@Jelie-VM:~$ env
SHELL=/bin/bash
SESSION_MANAGER=local/Jelie-VM:@/tmp/.ICE-unix/1283,unix/Jelie-VM:/tmp/.ICE-unix
/1283
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
SSH_AGENT_LAUNCHER=gnome-keyring
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
GTK_MODULES=gail:atk-bridge
PWD=/home/jelie
LOGNAME=jelie
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=wayland
SYSTEMD_EXEC_PID=1310
XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.RVE6M3
HOME=/home/jelie
USERNAME=jelie
IM_CONFIG_PHASE=1
```

Here, the ability to pass an environment variable to the printenv command to view its value is demonstrated. In this case, the PWD (present working directory) environment variable is used.

```
jelie@Jelie-VM:~$ printenv PWD
/home/jelie
jelie@Jelie-VM:~$
```

Here, the same result is achieved using the env command, but it requires piping the output and using grep to filter for the variable.

```
jelie@Jelie-VM:~$ env | grep PWD
PWD=/home/jelie
OLDPWD=/home/jelie/Desktop
jelie@Jelie-VM:~$
```

Command: export MYFIRSTENV=”My First ENV”

Command: unset MYFIRSTENV

Command explanation:

- **Export:** This command is used to set an environment variable.
- **MYFIRSTENV:** This is the name of the environment variable being created.
- **”My First ENV”:** This is the value assigned to the environment variable.
- **Unset:** This command removes an environment variable from the shell.
- **MYFIRSTENV:** This specifies the environment variable to be removed.

Command line output:

Here, the export command is used to create the first environment variable. Then, echo is used to print it to the console to confirm that it worked, making sure to include the $ before the variable name so that it retrieves the value of the variable. The unset command is then used to remove the variable, and it is checked again using the same echo command to confirm that it was removed.

```
jelie@Jelie-VM:~$ export MYFIRSTENV="My First ENV"
jelie@Jelie-VM:~$ echo $MYFIRSTENV
My First ENV
jelie@Jelie-VM:~$ unset MYFIRSTENV
jelie@Jelie-VM:~$ echo $MYFIRSTENV

jelie@Jelie-VM:~$
```

Task 2: Passing Environment Variables from Parent Process to Child Process:

In this task, we study how a child process gets its environment variables from its parent. In Unix, fork() creates a new process by duplicating the calling process. The new process, referred to as the child, is an exact duplicate of the calling process, referred to as the

parent; however, several things are not inherited by the child (please see the manual of fork() by typing the following command: man fork). In this task, we would like to know whether the parent's environment variables are inherited by the child process or not.

Step 1.

Please compile and run the following program and describe your observation. The program can be found in the Labsetup folder; it can be compiled using "gcc myprintenv.c", which will generate a binary called a.out. Let's run it and save the output into a file using "a.out > file"

Command: gcc myprintenv.c

Command explanation:

- **Gcc:** Stands for GNU Compiler Collections. This command is the Compiler used to compile C source code into an executable program.
- **myprintenv.c:** This is the C source file being compiled.

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
cap_leak.c  catall.c  myenv.c  myprintenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc myprintenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out  cap_leak.c  catall.c  myenv.c  myprintenv.c
```

Command: ./a.out > file

Command explanation:

- **./a.out:** This runs the compiled executable file generated by gcc.
- **>:** This is the output redirection operator. It redirects the output of the program into a file instead of displaying it in the terminal.
- **File:** This is the name of the file where the output of the redirect will be stored.

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ./a.out > file
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out  cap_leak.c  catall.c  file  myenv.c  myprintenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Step 2.

Now comment out the printenv() statement in the child process case (Line ①), and uncomment the printenv() statement in the parent process case (Line ②). Compile and run the code again, and describe your observation. Save the output in another file.

Command: nano myprintenv.c

Command explanation:

- **Nano:** This is a text editor used to open and modify files directly in the terminal.
- **Myprintenv.c:** This is the source file being edited.

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ nano myprintenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Here, the requested edits for this step have been completed.

```
  GNU nano 6.2                          myprintenv.c *
  int i = 0;
  while (environ[i] != NULL) {
     printf("%s\n", environ[i]);
     i++;
  }
}

void main()
{
  pid_t childPid;
  switch(childPid = fork()) {
    case 0:  /* child process */
      // printenv();
      exit(0);
    default:  /* parent process */
       printenv();
      exit(0);
  }
}


^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

Command: gcc printenv.c

Command: ./a.out > file2

Command explanation:

- **gcc:** This command recompiles the modified C source file into an executable program.
- **/a.out:** This runs the newly compiled program.
- **>:** This is the output redirection operator used to send the program output to a file.
- **file2:** This is the file where the new output is being stored.

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc myprintenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$ ./a.out > file2
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out  cap_leak.c  catall.c  file  file2  myenv.c  myprintenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Step 3.

Compare the difference of these two files using the diff command. Please draw your conclusion.

Command: diff file file2

Command explanation:

- **diff:** This command compares the contents of two files line by line and displays any differences between them.
- **file:** This is the first file being compared.
- **file2:** This is the second file being compared.

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ diff file file2
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Conclusion: After reading the output from the man fork command and performing this test, I believe that the parent's environment variables are passed on to the child process. As shown by the diff command, there is no difference in the output, meaning the environment variables are the same for both the parent process and its fork. This demonstrates that the child process inherits a copy of the parent's environment at the time of creation.

Task 3: Environment Variables and execve()

In this task, we study how environment variables are affected when a new program is executed via execve(). The function execve() calls a system call to load a new command and execute it; this function never returns. No new process is created; instead, the calling process's text, data, bss, and stack are overwritten by that of the program loaded. Essentially, execve() runs the new program inside the calling process. We are interested in what happens to the environment variables; are they automatically inherited by the new program?

Step 1.

Please compile and run the following program, and describe your observation. This program simply executes a program called /usr/bin/env, which prints out the environment variables of the current process.

Command: gcc myenv.c -o myenv

Command: ./myenv

Command explanation:

- **Gcc:** Stands for GNU Compiler Collections. This command is the Compiler used to compile C source code into an executable program.
- **myenv.c:** This is the C source file being compiled.
- **-o:** This argument indicates that the user will provide the name of the output executable file instead of using the default name a.out.
- **Myenv:** This argument specifies the name of the output executable file that will be created by the compiler.
- **./myenv:** This runs the compiled executable program from the current directory.

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out  cap_leak.c  catall.c  file  file2  myenv.c  myprintenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc myenv.c -o myenv
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out  cap_leak.c  catall.c  file  file2  myenv  myenv.c  myprintenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$ ./myenv
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Observation:

After compiling and running the program with execve("/usr/bin/env", argv, NULL);, the output does not display any environment variables. The last argument of the execve() command is used to pass the environment array, and when this field is NULL, it is as if there are no environment variables available to the new program.

Step 2.

Change the invocation of execve() in Line ① to the following; describe your observation.
execve("/usr/bin/env", argv, environ);

Command: nano myenv.c

Command explanation:

- **Nano:** This is a text editor used to open and modify files in the terminal.

- **Myenv.c:** This is the source file being edited.

Command line output:

```
  GNU nano 6.2                                 myenv.c *
#include <unistd.h>

extern char **environ;

int main()
{
  char *argv[2];

  argv[0] = "/usr/bin/env";
  argv[1] = NULL;

  execve("/usr/bin/env", argv, environ);

  return 0 ;
}
```

```
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

```
jelie@Jelie-VM:~/Desktop/Labsetup$ nano myenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc myenv.c -o myenv2
jelie@Jelie-VM:~/Desktop/Labsetup$ ./myenv2
SHELL=/bin/bash
SESSION_MANAGER=local/Jelie-VM:@/tmp/.ICE-unix/1283,unix/Jelie-VM:/tmp/.ICE-unix
/1283
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
SSH_AGENT_LAUNCHER=gnome-keyring
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
GTK_MODULES=gail:atk-bridge
PWD=/home/jelie/Desktop/Labsetup
LOGNAME=jelie
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=wayland
SYSTEMD_EXEC_PID=1310
XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.RVE6M3
HOME=/home/jelie
```

Conclusion: After changing the execve() call to execve("/usr/bin/env", argv, environ);, the output now displays all the environment variables. This is because environ contains the current process's environment variables, and passing it as the third argument to execve() explicitly provides those variables to the new program. Unlike using NULL, this allows the executed program to have access to the environment from the original process.

Step 3.

Please draw your conclusion regarding how the new program gets its environment variables.

Conclusion: Programs using the execve() function only receive environment variables if they are explicitly passed as the last argument. If NULL is passed, the new program does not inherit any environment variables. Therefore, environment variables are not automatically inherited by execve(); they must be provided explicitly through the execve() function call.

Task 4: Environment Variables and system()

In this task, we study how environment variables are affected when a new program is executed via the system() function. This function is used to execute a command, but unlike execve(), which directly executes a command, system() actually executes "/bin/sh -c command", i.e., it executes /bin/sh, and asks the shell to execute the command. If you look at the implementation of the system() function, you will see that it uses execl() to execute /bin/sh; execl() calls execve(), passing to it the environment variables array. Therefore, using system(), the environment variables of the calling process is passed to the new program /bin/sh. Please compile and run the following program to verify this.

#include <stdio.h>#include<stdlib.h> int main() { system("/usr/bin/env"); return 0 ; }

Step 1.

Command: nano task4.c

Command explanation:

- **Nano:**
- **Task4:**

Command line output:

```
  GNU nano 6.2                          task4.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
system("/usr/bin/env");
return 0;
}
```

```
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out  cap_leak.c  catall.c  file  file2  myenv  myenv2  myenv.c  myprintenv.c
jelie@Jelie-VM:~/Desktop/Labsetup$ nano task4.c
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out       catall.c  file2  myenv2   myprintenv.c
cap_leak.c  file      myenv  myenv.c  task4.c
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc task4.c -o task4
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out       catall.c  file2  myenv2   myprintenv.c  task4.c
cap_leak.c  file      myenv  myenv.c  task4
jelie@Jelie-VM:~/Desktop/Labsetup$ ./task4
LESSOPEN=| /usr/bin/lesspipe %s
USER=jelie
XDG_SESSION_TYPE=wayland
SHLVL=1
HOME=/home/jelie
OLDPWD=/home/jelie
DESKTOP_SESSION=ubuntu
GNOME_SHELL_SESSION_MODE=ubuntu
GTK_MODULES=gail:atk-bridge
SYSTEMD_EXEC_PID=1261
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
COLORTERM=truecolor
IM_CONFIG_PHASE=1
```

Observation:

After compiling and running the program, the output displays all the environment variables. This shows that the program executed using the system() function has access to the environment variables of the current process. Unlike the execve() case with NULL, the environment variables are still present and visible in the output.

Conclusion:

Programs using the system() function automatically inherit environment variables from the calling process. This is because system() executes /bin/sh, which then runs the command, and the environment variables are passed along to the new program. This is different from execve(), where environment variables must be explicitly provided.

Task 5: Environment Variable and Set-UID Programs

Set-UID is an important security mechanism in Unix operating systems. When a Set-UID program runs, it assumes the owner's privileges. For example, if the program's owner is root, when anyone runs this program, the program gains the root's privileges during its execution. Set-UID allows us to do many interesting things, but since it escalates the user's privilege, it is quite risky. Although the behaviors of Set-UID programs are decided by their program logic, not by users, users can indeed affect the behaviors via environment variables. To understand how Set-UID programs are affected, let us first figure out whether environment variables are inherited by the Set-UID program's process from the user's process.

Step 1.

Write the following program that can print out all the environment variables in the current process. #include <stdio.h>#include<stdlib.h> extern char **environ; int main() { int i = 0; while (environ[i] != NULL) { printf("%s\n", environ[i]); i++; } }

Command: nano foo.c

Command explanation:

- **nano:** Opens a text editor in the terminal to create or edit a file.
- **foo.c:** The C source file being created, which prints all environment variables.

Command line output:

```
  GNU nano 6.2                                foo.c *
#include <stdio.h>
#include <stdlib.h>

extern char **environ;
int main()
{
        int i = 0;
        while (environ[i] != NULL) {
                printf("%s\n",environ[i]);
                i++;
        }
}
```

```
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out       catall.c  file2  myenv2   myprintenv.c  task4.c
cap_leak.c  file      myenv  myenv.c  task4
jelie@Jelie-VM:~/Desktop/Labsetup$ nano foo.c
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out       catall.c  file2  myenv   myenv.c       task4
cap_leak.c  file      foo.c  myenv2  myprintenv.c  task4.c
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Step 2.

Compile the above program, change its ownership to root, and make it a Set-UID program.

// Asssume the program's name is foo $ sudo chown root foo $ sudo chmod 4755 foo

Command: gcc foo.c -o foo

Command: sudo chown root foo

Command: sudo chmod 4755 foo

Command explanation:

- **Gcc:** GNU Compiler Collection, compiles C source code into an executable.
- **Foo.c:** The source code file being compiled.
- **-o foo:** Names the output executable foo instead of a.out.

- **sudo:** Grants administrator privileges for commands that require root access.
- **chown:** Changes ownership of a file
- **root:** Specifies that the new owner of the file will be the root user.
- **foo:** The name of the file whose ownership is being changed.
- **sudo:** Grants administrator privileges for commands that require root access.
- **chmod:** Changes the permissions of a file or directory.
- **4755:** Sets the file mode to rwsr-xr-x — the leading 4 sets the Set-UID bit, 7 gives the owner read/write/execute, 5 gives group read/execute, and 5 gives others read/execute.
- **Foo:** The name of the file whose permissions are being modified.

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc foo.c -o foo
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out       catall.c  file2  foo.c  myenv2   myprintenv.c  task4.c
cap_leak.c  file      foo    myenv  myenv.c  task4
```

```
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chown root foo
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chmod 4755 foo
jelie@Jelie-VM:~/Desktop/Labsetup$ ls -l
total 112
-rwxrwxr-x 1 jelie jelie 16152 Mar 25 08:39 a.out
-rw-rw-r-- 1 jelie jelie   761 Dec 27  2020 cap_leak.c
-rw-rw-r-- 1 jelie jelie   471 Feb 19  2021 catall.c
-rw-rw-r-- 1 jelie jelie  2926 Mar 25 07:43 file
-rw-rw-r-- 1 jelie jelie  2894 Mar 25 08:40 file2
-rwsr-xr-x 1 root  jelie 16032 Mar 26 02:44 foo
-rw-rw-r-- 1 jelie jelie   159 Mar 26 02:44 foo.c
-rwxrwxr-x 1 jelie jelie 16016 Mar 25 09:41 myenv
-rwxrwxr-x 1 jelie jelie 16080 Mar 25 10:01 myenv2
-rw-rw-r-- 1 jelie jelie   183 Mar 25 10:01 myenv.c
-rw-rw-r-- 1 jelie jelie   420 Mar 25 08:37 myprintenv.c
-rwxrwxr-x 1 jelie jelie 15960 Mar 26 02:17 task4
-rw-rw-r-- 1 jelie jelie    89 Mar 26 02:16 task4.c
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Step 3.

In your shell (you need to be in a normal user account, not the root account), use the export command to set the following environment variables (they may have already exist): • PATH • LD LIBRARY PATH • ANY NAME (this is an environment variable defined by you, so pick whatever name you want). These environment variables are set in the user's shell process. Now, run the Set-UID program from Step 2 in your shell. After you type the name of the

program in your shell, the shell forks a child process, and uses the child process to run the program. Please check whether all the environment variables you set in the shell process (parent) get into the Set-UID child process. Describe your observation. If there are surprises to you, describe them.

Command: export PATH="$HOME/custom_bin:$PATH"

Command: export LD_LIBRARY_PATH="$/home/jelie /Desktop"

Command: export ANY_NAME="This is my env"

Command: ./foo | grep -E '^(PATH|LD_LIBRARY_PATH|ANY_NAME)='

Command explanation:

- **export PATH="$HOME/custom_bin:$PATH":** Adds the directory $HOME/custom_bin to the front of the existing PATH environment variable. This ensures that executables in $HOME/custom_bin are found first when running commands.
- **export LD_LIBRARY_PATH="/home/jelie/Desktop":** Sets the LD_LIBRARY_PATH environment variable to /home/jelie/Desktop, which specifies additional directories where dynamic libraries are searched for.
- **export ANY_NAME="This is my env":** Creates a custom environment variable called ANY_NAME with the value "This is my env".
- **./foo | grep -E '^(PATH|LD_LIBRARY_PATH|ANY_NAME)=':** Runs the foo Set-UID program and pipes its output to grep. The -E flag enables extended regular expressions, and the pattern

Command line output:

Original env values:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/loc
al/games:/snap/bin:/snap/bin
jelie@Jelie-VM:~/Desktop/Labsetup$ echo $LD_LIBRARY_PATH

jelie@Jelie-VM:~/Desktop/Labsetup$ echo $ANY_NAME
```

Modified env values:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ export ANY_NAME="This is my env"
jelie@Jelie-VM:~/Desktop/Labsetup$ export LD_LIBRARY_PATH="/home/jelie/Desktop"
jelie@Jelie-VM:~/Desktop/Labsetup$ echo $ANY_NAME
This is my env
jelie@Jelie-VM:~/Desktop/Labsetup$ echo $LD_LIBRARY_PATH
/home/jelie/Desktop
jelie@Jelie-VM:~/Desktop/Labsetup$ export PATH="$HOME/custom_bin:$PATH"
jelie@Jelie-VM:~/Desktop/Labsetup$ echo $PATH
/home/jelie/custom_bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/
bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
jelie@Jelie-VM:~/Desktop/Labsetup$
```

This is the output of a piped grep looking for the 3 environment variables:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ./foo | grep -E '^(PATH|LD_LIBRARY_PATH|ANY_N
AME)='
ANY_NAME=This is my env
PATH=/home/jelie/custom_bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/s
bin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
```

Conclusion: The Set-UID program inherited the PATH and custom environment variable ANY_NAME from the parent shell, showing that normal variables can be passed to Set-UID programs. However, LD_LIBRARY_PATH did not appear in the child process, which is expected because the system ignores potentially dangerous environment variables for security reasons. Although I modified PATH, it was still passed to the Set-UID program, which shows that the kernel allows some environment variables like PATH to be inherited even for Set-UID programs.

Task 6 The PATH Environment Variable and Set-UID Programs

Because of the shell program invoked, calling system() within a Set-UID program is quite dangerous. This is because the actual behavior of the shell program can be affected by environment variables, such as PATH; these environment variables are provided by the user, who may be malicious. By changing these variables, malicious users can control the behavior of the Set-UID program. In Bash, you can change the PATH environment variable in the following way (this example adds the directory /home/seed to the beginning of the PATH environment variable): $ export PATH=/home/seed:$PATH The Set-UID program below is supposed to execute the /bin/ls command; however, the programmer only uses the relative path for the ls command, rather than the absolute path:

Step 1.

.

Command:

Command explanation:

- **nano:** Opens a text editor in the terminal to create or edit files.
- **task6.c:** This is the C source file being created for this task.

Command line output:

```
  GNU nano 6.2                          task6.c *
int main()
{
        system("ls");
        return 0;
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

Command: gcc task6.c -o task6

Command explanation:

- **gcc:** Compiles the C source code into an executable program.
- **task6.c:** The source file being compiled.
- **-o task6:** Names the output executable task6.

Command line output:

Linux is giving a waning here about using the system function

```
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc task6.c -o task6
task6.c: In function ‘main’:
task6.c:3:9: warning: implicit declaration of function ‘system’ [-Wimplicit-func
tion-declaration]
    3 |         system("ls");
      |         ^~~~~~
```

Command: sudo chown root task6

Command: sudo chmod 4755 task6

Command explanation:

- **sudo:** Temporarily grants administrator privileges.
- **chown root task6:** Changes ownership of the file to root.
- **chmod 4755 task6:** Sets the Set-UID bit so the program runs with root privileges.

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chown root task6
jelie@Jelie-VM:~/Desktop/Labsetup$ ls -l task6
-rwxrwxr-x 1 root jelie 15960 Mar 26 06:11 task6
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chmod 4755 task6
jelie@Jelie-VM:~/Desktop/Labsetup$ ls -l task6
-rwsr-xr-x 1 root jelie 15960 Mar 26 06:11 task6
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Step 2.

Create malicious ls

Command: nano ls

Command explanation:

- **nano ls:** Creates a file named ls which will act as a fake command.
- **chmod +x ls:** Makes the file executable.

Command line output:


```
jelie@Jelie-VM:~/Desktop/Labsetup$ nano ls
jelie@Jelie-VM:~/Desktop/Labsetup$ chmod +x ls
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Step 3.

Modify PATH to prioritize malicious file and Run the Set-UID program

Command: export PATH="$PWD:$PATH"

Command explanation:

- **export PATH="$PWD:$PATH":** Adds the current directory to the beginning of the PATH so the system will find the fake ls before /bin/ls.

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ export PATH="$PWD:$PATH"
jelie@Jelie-VM:~/Desktop/Labsetup$ ./task6
I am malicius!!
jelie
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Conclusion: The Set-UID program executed the fake ls file instead of /bin/ls. This shows that modifying the PATH variable can change which program is executed when a relative

path is used. However, the output of whoami shows the current user instead of root. This is because /bin/sh (linked to dash) drops privileges when executed in a Set-UID program.

Step 4.

Link /bin/sh to zsh and remove protection

Command: sudo ln -sf /bin/zsh /bin/sh

Command explanation:

-

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo ln -sf /bin/zsh /bin/sh
[sudo] password for jelie:
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Observation: After modifying the PATH variable and running the Set-UID program, the malicious ls file was executed instead of the real /bin/ls. This is shown by the output "I am malicious!!", confirming that the program used the version of ls in the current directory due to the modified PATH. To check privileges, I used the id command inside the malicious script. The output showed: uid=1000(jelie) gid=1000(jelie) This indicates that both the real user ID (ruid) and effective user ID (euid) are still jelie, not root.

Conclusion: The attack successfully demonstrates that modifying the PATH environment variable can control which program is executed in a Set-UID program when relative paths are used. However, the malicious code did not run with root privileges. This shows that modern systems implement security protections that prevent privilege escalation, even when a Set-UID program is vulnerable to a PATH attack. This highlights why using system() with relative paths is dangerous, but also how operating systems attempt to mitigate these risks.

Reset the path so ls can be used
export
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin"

Task 7: The LD PRELOAD Environment Variable and Set-UID Programs

In this task, we study how Set-UID programs deal with some of the environment variables. Several environment variables, including LD PRELOAD, LD LIBRARY PATH, and other LD * influence the behavior of dynamic loader/linker. A dynamic loader/linker is the part of an

operating system (OS) that loads (from persistent storage to RAM) and links the shared libraries needed by an executable at run time. In Linux, ld.so or ld-linux.so, are the dynamic loader/linker (each for different types of binary). Among the environment variables that affect their behaviors, LD LIBRARY PATH and LD PRELOAD are the two that we are concerned in this lab. In Linux, LD LIBRARY PATH is a colon-separated set of directories where libraries should be searched for first, before the standard set of directories. LD PRELOAD specifies a list of additional, user-specified, shared libraries to be loaded before all others. In this task, we will only study LD PRELOAD.

Step 1.

First, we will see how these environment variables influence the behavior of dynamic loader/linker when running a normal program. Please follow these steps:

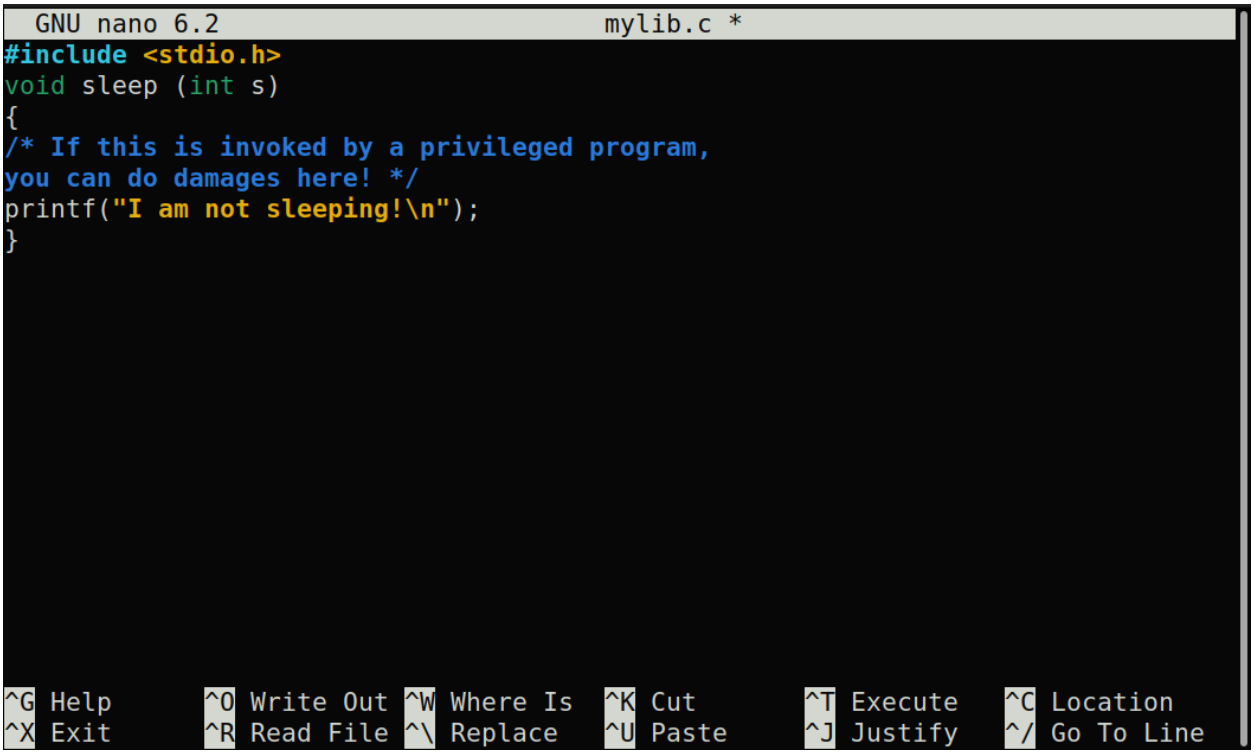
1. Let us build a dynamic link library. Create the following program, and name it mylib.c. It basically overrides the sleep() function in libc.

Command: **nano mylib.c**

Command explanation:

- **nano:** Opens a text editor to create or edit files.
- **mylib.c:** This is the C file that overrides the sleep() function.

Command line output:



2 We can compile the above program using the following commands (in the -lc argument, the second character is l`):

Command: **gcc -fPIC -g -c mylib.c**

Command: **gcc -shared -o libmylib.so.1.0.1 mylib.o -lc**

Command explanation:

- :
- :

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ nano mylib.c
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc -fPIC -g -c mylib.c
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc -shared -o libmylib.so.1.0.1 mylib.o -lc
jelie@Jelie-VM:~/Desktop/Labsetup$
```

3. Now, set the LD PRELOAD environment variable: $ export

Command: **export LD_PRELOAD=./libmylib.so.1.0.1**

Command explanation:

- **export:** Sets an environment variable.
- **LD_PRELOAD:** this env forces the system to load this shared library before others.
- **./libmylib.so.1.0.1:** The custom library that overrides sleep().

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$  export LD_PRELOAD=./libmylib.so.1.0.1
jelie@Jelie-VM:~/Desktop/Labsetup$
```

4. Finally, compile the following program myprog, and in the same directory as the above dynamic link library libmylib.so.1.0.1:

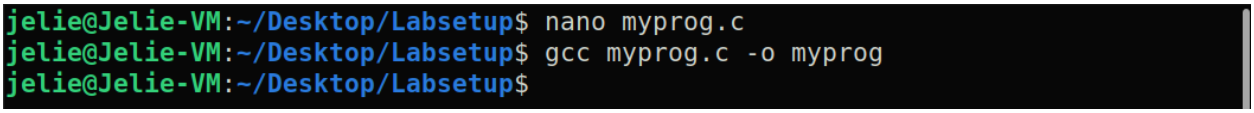
Command: **nano myprog.c**

Command: **gcc myprog.c -o myprog**

Command explanation:

- **myprog.c:** Program that calls sleep
- **-o myprog:** Names the executable myprog

Command line output:



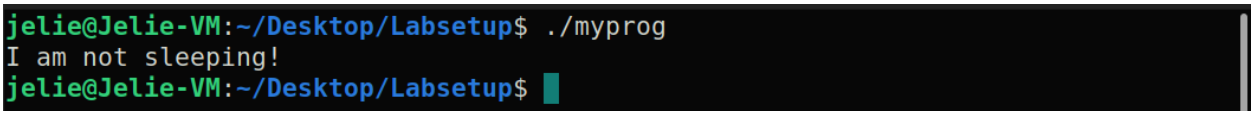
Step 2.

After you have done the above, please run myprog under the following conditions, and observe what happens. SEED Labs – Environment Variable and Set-UID Program Lab 7

• Make myprog a regular program, and run it as a normal user.

Command: ./myprog

Command line output:



Observation: The custom library is loaded, and the sleep() function is overridden. The program does not actually sleep.

• Make myprog a Set-UID root program, and run it as a normal user.

Command: sudo chown root myprog

Command: sudo chmod 4755 myprog

Command: ./myprog

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chown root myprog
[sudo] password for jelie:
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chmod 4755 myprog
jelie@Jelie-VM:~/Desktop/Labsetup$ ls -l myprog
-rwsr-xr-x 1 root jelie 15960 Mar 26 08:10 myprog
jelie@Jelie-VM:~/Desktop/Labsetup$ ./myprog
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Observation: The LD_PRELOAD variable is ignored, and the original sleep() function is used.

• Make myprog a Set-UID root program, export the LD PRELOAD environment variable again in the root account and run it.

Command: sudo su

Command: export LD_PRELOAD=./libmylib.so.1.0.1

Command: ./myprog

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo su
root@Jelie-VM:/home/jelie/Desktop/Labsetup# export LD_PRELOAD=./libmylib.so.1.0.
1
root@Jelie-VM:/home/jelie/Desktop/Labsetup# ./myprog
I am not sleeping!
root@Jelie-VM:/home/jelie/Desktop/Labsetup#
```

Observation: When LD_PRELOAD is set by root, the library is loaded and overrides sleep().

• Make myprog a Set-UID user1 program (i.e., the owner is user1, which is another user account), export the LD PRELOAD environment variable again in a different user's account (not-root user) and run it.

Command: sudo adduser lab2user

Command: sudo chown lab2user myprog

Command: sudo chmod 4755 myprog

Command: export LD_PRELOAD=./libmylib.so.1.0.1

Command: ./myprog

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chown lab2user myprog
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chmod 4755 myprog
jelie@Jelie-VM:~/Desktop/Labsetup$ export LD_PRELOAD=./libmylib.so.1.0.1
jelie@Jelie-VM:~/Desktop/Labsetup$ ./myprog
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Observation: The LD_PRELOAD variable is ignored when set by a different user.

Step 3.

Conclusion The behavior differs depending on the privilege level of the program and who sets the LD_PRELOAD variable. For most programs, LD_PRELOAD is allows overriding of functions like sleep(). However, for Set-UID programs, the system ignores LD_PRELOAD when it is set by a non-privileged user. This is a security feature.

Task 8: Invoking External Programs Using system() versus execve()

Although system() and execve() can both be used to run new programs, system() is quite dangerous if used in a privileged program, such as Set-UID programs. We have seen how the PATH environment variable affect the behavior of system(), because the variable affects how the shell works. execve() does not have the problem, because it does not invoke shell. Invoking shell has another dangerous consequence, and this time, it has nothing to do with environment variables. Let us look at the following scenario. Bob works for an auditing agency, and he needs to investigate a company for a suspected fraud. For the investigation purpose, Bob needs to be able to read all the files in the company's Unix system; on the other hand, to protect the integrity of the system, Bob should not be able to modify any file. To achieve this goal, Vince, the superuser of the system, wrote a special set-root-uid program (see below), and then gave the executable permission to Bob. This program requires Bob to type a file name at the command line, and then it will run /bin/cat to display the specified file. Since the program is running as a root, it can display any file Bob specifies. However, since the program has no write operations, Vince is very sure that Bob cannot use this special program to modify any file.

Step 1.

Compile the above program, make it a root-owned Set-UID program. The program will use system() to invoke the command. If you were Bob, can you compromise the integrity of the system? For example, can you remove a file that is not writable to you?.

Command: gcc catall.c -o catall

Command: sudo chown root catall

Command: sudo chmod 4755 catall

Command explanation:

- **gcc: Compiles the program**
- **-o catall: Names the executable catall**
- **sudo: Grants admin privileges**
- **chown root catall: Makes root the owner**
- **chmod 4755 catall: Sets Set-UID so it runs as root**

Command line output:

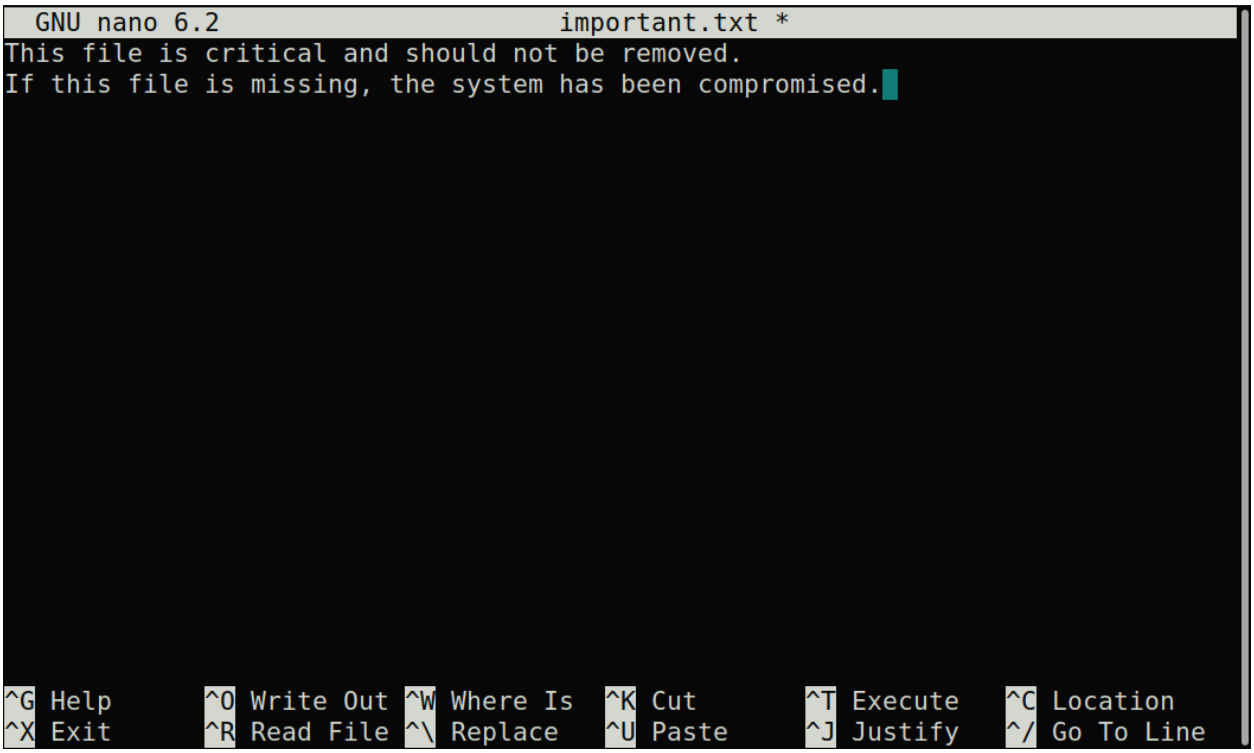
```
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc catall.c -o catall
jelie@Jelie-VM:~/Desktop/Labsetup$ ls -l catall
-rwxrwxr-x 1 jelie jelie 16184 Mar 26 08:59 catall
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chown root catall
[sudo] password for jelie:
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chmod 4755 catall
jelie@Jelie-VM:~/Desktop/Labsetup$ ls -l catall
-rwsr-xr-x 1 root jelie 16184 Mar 26 08:59 catall
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Command: nano important.txt

Command: ./catall "important.txt"

Command: ./catall "important.txt; rm important.txt "

Command line output:




```
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out       file   libmylib.so.1.0.1  myenv.c       myprog    task6
cap_leak.c  file2  ls                 mylib.c       myprog.c  task6.c
catall      foo    myenv              mylib.o       task4
catall.c    foo.c  myenv2             myprintenv.c  task4.c
jelie@Jelie-VM:~/Desktop/Labsetup$ nano important.txt
jelie@Jelie-VM:~/Desktop/Labsetup$ ./catall "important.txt"
This file is critical and should not be removed.
If this file is missing, the system has been compromised.
jelie@Jelie-VM:~/Desktop/Labsetup$ ./catall "important.txt;rm important.txt"
This file is critical and should not be removed.
If this file is missing, the system has been compromised.
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out       file   libmylib.so.1.0.1  myenv.c       myprog    task6
cap_leak.c  file2  ls                 mylib.c       myprog.c  task6.c
catall      foo    myenv              mylib.o       task4
catall.c    foo.c  myenv2             myprintenv.c  task4.c
jelie@Jelie-VM:~/Desktop/Labsetup$ ./catall "important.txt"
/bin/cat: important.txt: No such file or directory
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Observation: The program is vulnerable, by adding ; rm filename, I was able to execute additional commands. Since the program runs with root privileges, the injected command also runs as root, allowing me to delete files I normally do not have permission to modify.

Step 2.

Comment out the system(command) statement, and uncomment the execve() statement; the program will use execve() to invoke the command. Compile the program, and make it a root-owned Set-UID. Do your attacks in Step 1 still work? Please describe and explain your observations..

Command: nano catall.c

Command explanation:

- :

Command line output:

```
  GNU nano 6.2                          catall.c *
  char *v[3];
  char *command;

  if(argc < 2) {
    printf("Please type a file name.\n");
    return 1;
  }

  v[0] = "/bin/cat"; v[1] = argv[1]; v[2] = NULL;

  command = malloc(strlen(v[0]) + strlen(v[1]) + 2);
  sprintf(command, "%s %s", v[0], v[1]);

  // Use only one of the followings.
  // system(command);
  execve(v[0], v, NULL);

  return 0 ;
}


^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

Command: gcc catall.c -o catallSafe

Command: sudo chown root catallSafe

Command: sudo chmod 4755 catallSafe

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ nano catall.c
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc catall.c -o catallSafe
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chown root catallSafe
[sudo] password for jelie:
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chmod 4755 catallSafe
jelie@Jelie-VM:~/Desktop/Labsetup$ ls -l catallSafe
-rwsr-xr-x 1 root jelie 16184 Mar 26 09:54 catallSafe
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Command: ./catallSafe "important.txt"

Command: ./catallSafe "important.txt; rm important.txt"

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ./catallSafe "important.txt"
This file is critical and should not be removed.
If this file is missing, the system has been compromised.
jelie@Jelie-VM:~/Desktop/Labsetup$ ./catallSafe "important.txt; rm important.txt
"
/bin/cat: 'important.txt; rm important.txt': No such file or directory
jelie@Jelie-VM:~/Desktop/Labsetup$
```

observation: The attack no longer works. The entire input is treated as a single filename instead of being interpreted as multiple commands.

Conclusion: The execve() function does not invoke a shell, and does not interpret special characters like ;. This prevents command injection attacks. Therefore, execve() is much safer than system() when writing Set-UID programs.

Task 9: Capability Leaking

To follow the Principle of Least Privilege, Set-UID programs often permanently relinquish their root privileges if such privileges are not needed anymore. Moreover, sometimes, the program needs to hand over its control to the user; in this case, root privileges must be revoked. The setuid() system call can be used to revoke the privileges. According to the manual, “setuid() sets the effective user ID of the calling process. If the effective UID of the caller is root, the real UID and saved set-user-ID are also set”. Therefore, if a Set-UID program with effective UID 0 calls setuid(n), the process will become a normal process, with all its UIDs being set to n. When revoking the privilege, one of the common mistakes is capability leaking. The process may have gained some privileged capabilities when it was still privileged; when the privilege is downgraded, if the program does not clean up those capabilities, they may still be accessible by the non-privileged process. In other words, although the effective user ID of the process becomes non-privileged, the process is still privileged because it possesses privileged capabilities. Compile the following program, change its owner to root, and make it a Set-UID program. Run the program as a normal

user. Can you exploit the capability leaking vulnerability in this program? The goal is to write to the /etc/zzz file as a normal user.

Step 1.

.

Command: gcc cap_leak.c -o cap_leak

Command: sudo chown root cap_leak

Command: sudo chmod 4755 cap_leak

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ls
a.out       catallSafe  foo.c              myenv    mylib.o       task4
cap_leak.c  file        important.txt      myenv2   myprintenv.c  task4.c
catall      file2       libmylib.so.1.0.1  myenv.c  myprog        task6
catall.c    foo         ls                 mylib.c  myprog.c      task6.c
jelie@Jelie-VM:~/Desktop/Labsetup$ gcc cap_leak.c -o cap_leak
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chown root cap_leak
[sudo] password for jelie:
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chmod 4755 cap_leak
jelie@Jelie-VM:~/Desktop/Labsetup$ ls -l cap_leak
-rwsr-xr-x 1 root jelie 16272 Mar 26 10:10 cap_leak
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Command: sudo touch /etc/zzz

Command: sudo chmod 644 /etc/zzz

Command: sudo chown root:root cap_leak

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo touch /etc/zzz
jelie@Jelie-VM:~/Desktop/Labsetup$ cat /etc/zzz
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chmod 644 /etc/zzz
jelie@Jelie-VM:~/Desktop/Labsetup$ sudo chown root:root cap_leak
jelie@Jelie-VM:~/Desktop/Labsetup$ cat /etc/zzz
jelie@Jelie-VM:~/Desktop/Labsetup$ ls -l /etc/zzz
-rw-r--r-- 1 root root 0 Mar 26 10:14 /etc/zzz
jelie@Jelie-VM:~/Desktop/Labsetup$
```

Command: ./cap_leak

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ./cap_leak
fd is 3
$
```

Observation:The program prints a file descriptor fd is 3 A shell is launched after privileges are dropped

Command: **echo "HACKED" >&3**

Command: **cat /etc/zzz**

Command line output:

```
jelie@Jelie-VM:~/Desktop/Labsetup$ ./cap_leak
fd is 3
  echo "HACKED" >&3
  cat /etc/zzz
task 9
HACKED
$
```

Observation: The text "HACKED" is successfully written to /etc/zzz

Conclusion: the program is vulnerable to capability leaking. The file /etc/zzz is opened with root privileges and spawned a root shell This allows a normal user to write to a protected file.

3. Submission:

3.1.

You need to submit a detailed lab report, with screenshots, to describe what you have done and what you have observed. You also need to provide explanation to the observations that are interesting or surprising. Please also list the important code snippets followed by explanation. Simply attaching code without any explanation will not receive credits.